Attention Kernel Portability Across Six GPUs and Two Inference Regimes: A Measurement Study

Srivatsan Sarvesan
University of California, San Diego
ssarvesan@ucsd.edu

Abstract

We study whether an attention backend selected on one GPU remains a good choice on another. We benchmark forward prefill and single-token KV-cache decode across six NVIDIA GPUs using fixed library versions. A backend meets our separation criterion when the runner-up is at least 10% slower and the lower bound of a 95% bootstrap interval on the latency ratio exceeds 1. This criterion is met in 128/447 forward-prefill and 409/1,203 decode configurations. Among separated comparisons across Ampere, Ada and Hopper GPUs, the selected backend changes in 27/64 prefill comparisons and 29/289 decode comparisons. Across costed source-to-target transfers, median latency ratios are 1.002× for prefill and 1.000× for decode; the corresponding 95th percentiles are 1.610× and 1.325×. Some source choices have no eligible target timing and are reported separately. The results characterise backend selection under the tested workloads and software environments; they do not isolate architecture from host and driver effects.

1 Introduction

A practitioner choosing an attention kernel reads a benchmark run on one GPU. Does its winner survive a change of device?

The question is usually assumed rather than measured. Serving systems fix an attention backend by preference order and carry that order onto every device they deploy on. We ask whether that order holds: if the ordering belongs to the kernel, carrying it across devices is sound; if it belongs to the pairing of kernel and device, a preference order tuned on one part may not hold on another.

We time 10 prefill and 6 decode implementations over 403 configurations on the six GPUs of Table 1; 73,230 attempt records collapse to 11,835 usable (GPU, configuration, implementation) medians.[1] Every claim is a within-device ordering compared across devices, never a claim that one device is faster than another.

Two contributions follow from that. The first is a protocol that records which kernel actually ran rather than which was requested, and that reports a winner only where it is separated from the runner-up. The second is the finding itself: a backend choice should be qualified by the GPU, the regime and the library versions it was measured under.

2 Background and related work

FlashAttention (Dao et al., 2022) keeps the $N \times N$ scores out of HBM, on A100; FlashAttention-2 (Dao, 2024) is evaluated on A100 and defers Hopper; FlashAttention-3 (Shah et al., 2024) is that deferred work, evaluated on H100 only. Each reports its result on the one part its authors measured, which is the practice this study tests. FlashDecoding (Dao et al., 2023) adds split-KV for $q_{\text{len}} = 1$ on A100, and FlashInfer (Ye et al., 2025) generalises it into a serving engine, on A100 and H100.

Evaluation inherits the habit. The PyTorch 2 paper (Ansel et al., 2024) evaluates TorchInductor, which generates Triton (Tillet et al., 2019), on one A100; KernelBench (Ouyang et al., 2025) profiles 250 PyTorch workloads on an L40S; TritonBench (Li et al., 2025) evaluates 184 Triton operators on the A100. Serving systems fix the kernel by preference order rather than by measurement: vLLM (Kwon et al., 2023), Sarathi-Serve (Agrawal et al., 2024) and DistServe (Zhong et al., 2024) on A100, Splitwise (Patel et al., 2024) on DGX-A100 and DGX-H100. TVM (Chen et al., 2018) and Ansor

[1] Harness, analysis and figure code: https://github.com/Srivatsan6923/attention-kernel-portability. Measurements for this release are available under https://github.com/Srivatsan6923/attention-kernel-portability/releases/tag/v1.1, with SHA-256 checksums and instructions for regenerating the results.

(Zheng et al., 2020) instead search per target, conceding that the right implementation is a property of the target. Outside ML the question is mature: Williams et al. (2009) give the per-architecture ceiling, and Pennycook et al. (2016) and Deakin et al. (2019) define performance portability over a stated platform set and apply it across twelve platforms.

3 Study design

Implementations and grid. The prefill set is a naive reference, a `torch.compile` path in three variants, the three PyTorch SDPA backends (memory-efficient, flash, cuDNN), a vendored Triton tutorial kernel with a local BF16 adaptation, FlashAttention 2 and 3. The decode set is a naive KV-cache loop, an Inductor path, SDPA, `flash_attn_with_kvcache`, FlashInfer, and FlashAttention at query length 1. A *cell* keys regime, batch, query and KV head counts, head dimension, sequence length, dtype, forward or forward-plus-backward, launch mode, cache state and causality; the union grid is 159 prefill and 244 decode cells, with per-device coverage in Table 1.

Timing protocol. Backend order is randomised for each configuration, and each backend's timing blocks are then collected consecutively; the harness does not alternate backends between blocks, so we do not claim host or thermal effects are common-mode across a comparison. For execution without CUDA Graphs, hereafter eager, each block's elapsed CUDA-event time is divided by its call count; we take the median across blocks within a process, then the median across processes. Intervals come from a 2000-draw bootstrap that resamples whole process launches, since calls inside one launch share clocks and thermal point. Blocks are sized to target roughly 200 μs. Where a cell asks for a cold cache the L2 buffer is cleared once before each block, so a block of more than one call is not uniformly cold. CUDA-graph cells are timed by Triton's `do_bench_cudagraph`, whose `rep` argument is a millisecond budget rather than a repetition count. Between-process CV is a median 0.011 on A10 (n=1871) and 0.026 on H100 (n=2474). Processes are pinned to the NUMA node local to the GPU: two runs of the H100 decode grid at one commit, differing only in pinning and restricted to the eager warm cells, give a median between-process CV of 0.1116 unpinned against 0.0341 pinned over 966 (cell, implementation) triples spanning 192 cells on each side; the share of those triples with a CV above 25% falls from 30.7% to 9.9%. Both come from the `block_bench` path, which brackets a block of calls with a CUDA-event pair; they are dispersion across process launches rather than the per-call figure quoted above, and the two are not directly comparable.

Definitions. Let $t_{g,c,b,r}$ be the median attention-call latency inside process launch r, for GPU g, configuration c and backend b. The reported latency is the median over launches,

$$T_{g,c,b} \;=\; \operatorname*{median}_{r}\; t_{g,c,b,r}. \tag{1}$$

Writing b_1 and b_2 for the fastest and second-fastest eligible backends at (g, c), the separation ratio and the rule are

$$R_{g,c} \;=\; \frac{T_{g,c,b_2}}{T_{g,c,b_1}}, \tag{2}$$

$$\mathrm{Sep}(g,c) \;=\; \mathbf{1}\big[\,R_{g,c} \geq 1.10 \;\wedge\; L_{95}(R_{g,c}) > 1\,\big], \tag{3}$$

where L_{95} is the lower endpoint of a 95% bootstrap interval on the same ratio, resampling whole process launches. Both conditions are required. A configuration that fails the rule has not been shown to have equal backends; it is one where no fastest backend was established. Equations 1 and 2 are computed over the same launches: the fastest and second-fastest backends ran in identical launch sets in all 2,082 cells here, so no restriction to a shared subset was needed. The wider scan over every implementation pair, which produces the inversion counts, does need one. In 50 of 29,304 pairs, all of them on the RTX 5090 and all involving the naive reference, one backend lost a launch to an out-of-memory failure; those pairs are ranked and bootstrapped on the launches the two backends share.

Let $\mathcal{B}_{g,c}$ be the backends with a usable measurement at (g, c) and $b^*_{g,c}$ the winner. Over a set $\mathcal{E}$ of matched comparisons, each a triple (g, h, c) for which c was measured on both GPUs and $\mathrm{Sep}(g, c) = \mathrm{Sep}(h, c) = 1$,

$$\mathrm{WCR} \;=\; \frac{\sum_{(g,h,c)\in\mathcal{E}} \mathbf{1}\left[b^*_{g,c} \neq b^*_{h,c}\right]}{|\mathcal{E}|}. \tag{4}$$

The common-backend variant restricts each comparison to $\mathcal{B}^{\mathrm{common}}_{g,h,c} = \mathcal{B}_{g,c} \cap \mathcal{B}_{h,c}$, intersected per matched configuration rather than per regime, and re-derives b^* and Sep inside that set.

GPU	cc	family	BW (GB/s)	planned repeats pre/dec	cells pre/dec
A100	8.0	Ampere	1732.8	5/5	144/192
A10	8.6	Ampere	483.3	5/5	144/191
L40	8.9	Ada	644.4	3/3	144/192
L40S	8.9	Ada	644.4	5/5	144/192
H100	9.0	Hopper	2979.7	5/5	159/244
RTX 5090	12.0	Blackwell	1517.5	5/5	144/192

Table 1: GPU configurations and measurement coverage. Bandwidth is measured using an in-process copy benchmark on that host, never a vendor figure. *Planned repeats* are independent process launches, prefill/decode. Five process repeats were planned per configuration, except L40 with three. The median number of eligible repeats on L40 was two. *Cells* counts configurations with at least one eligible measurement.

Finally, the cost of carrying GPU g's choice to GPU h is

$$C_{g \to h,\, c} \;=\; \frac{T_{h,c,\, b^*_{g,c}}}{\min_{b \in \mathcal{B}_{h,c}} T_{h,c,b}}. \qquad (5)$$

A value of 1.20 means the source GPU's selected backend takes 20% longer than the fastest backend measured on the target. Where $b^*_{g,c} \notin \mathcal{B}_{h,c}$ the ratio is undefined; those cases are reported as a coverage count and never replaced by the target's own best.

Correctness gate. Every implementation is checked against a reference in its equivalence class, and only rows with a passing verdict for their own device and class enter a ranking. Of 73,230 attempt records, 54,213 (74.0%) completed with status `OK`; of those, 54,098 carry a passing verdict and are eligible, and 115 belong to one class that was never gated on its own device and are excluded; 16,326 are `UNSUPPORTED`, 1,590 were refused in advance by an out-of-memory predictor, and 950 raised a run-time error, all of them one implementation on one device (Section 6). The remaining 151 hit the allocator despite passing the predictor, a defect in the predictor and not in the implementation it was meant to protect. No evaluated correctness check failed. All claims are within-device orderings compared across devices, never that device A is faster than device B, and bandwidth denominators are measured on each host, not taken from a datasheet.

4 Dispatch validation

A latency is evidence about a kernel only if that kernel is the one that ran. We attempted to capture launched kernel names for each timed row; readable traces were obtained for 36,054 of 54,213 `OK` attempts. Coverage below is over all 54,213 `OK` attempts, not the 54,098 eligible rows. Of those 54,213, 36,054 (66.5%) captured a readable trace. Grouping by the tuple that determines dispatch, (implementation, GPU, head dimension, dtype, GQA ratio, mode, launch), gives 651 dispatch classes, 625 of them (96.0%) with at least one probed row. The class is a backend-family unit and certifies only that: within a class the template specialisation does vary with batch, and 156 of the flash-family classes launch both `flash_fwd_splitkv_combine` and `flash_fwd`, while the family the row was asked for is the same in every one of them. Class coverage is therefore not shape coverage, and the per-row figure above is the one to read for that. Among probed rows the requested kernel family is absent from the trace in 162 cases, a mismatch rate of 0.449%, or 0.299% over every audited row. The unprobed 33.5% count as unknown, never as agreement. Mismatches concentrate: 66 of the 162 are the SDPA memory-efficient backend on H100. The cells behind the headline comparisons are probed at the dataset's own rate: of the 551 device-cells entering the restricted columns of Table 2, 63.5% of passing rows carry a trace, and 51 of the 162 mismatches land on one of them.

The Inductor result needs its exact scope. Across 17,537 Inductor rows, 3,645 captured TorchInductor's `fuse_attention` counter, and on all 3,645, on all six devices, that counter is 0. Inductor's own attention-fusion pattern did not fire on any row whose counter we could read. The stronger claim, that no Inductor row reached a flash kernel at all, does not hold: 45 probed Inductor rows, all on H100, over 6 distinct cells, do carry a flash kernel in their trace.

5 Portability

Most cells have no decisive winner. We call a backend the *separated* winner of a configuration when it is the fastest backend meeting both the 10% margin and the confidence criterion: the runner-up's median latency is at least 1.10 times its own, and the lower bound of a 95% bootstrap interval on that ratio exceeds 1. Only 28.6% of forward-prefill and 34.0% of decode configurations have a separated winner, 128 of 447 and 409 of 1203, and Figure 1 gives the rate per device. A flip between implementations within a few percent

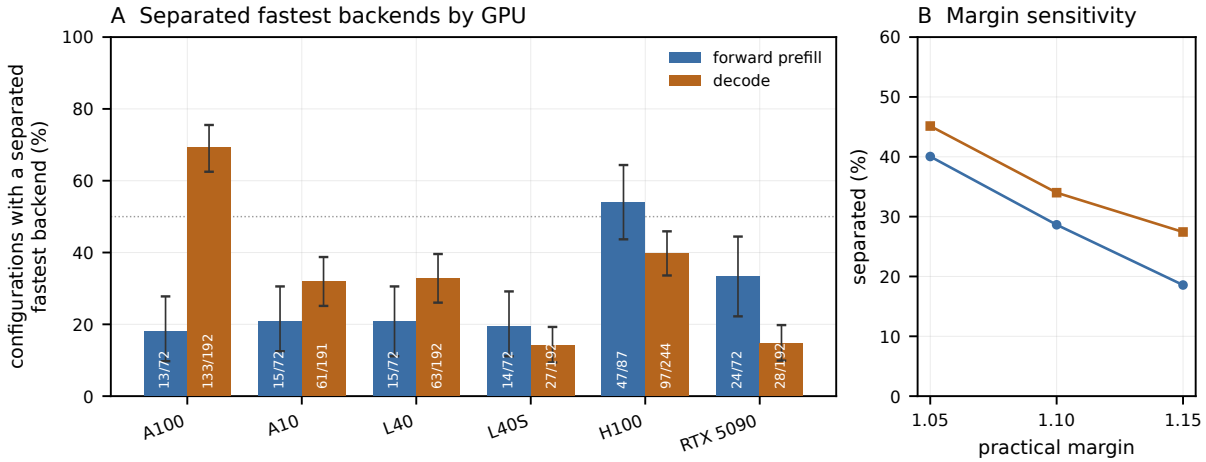

Figure 1: How often a fastest backend is separated from the runner-up, by both a 10% margin on the runner-up/fastest ratio and a cluster-bootstrap interval on that ratio excluding 1. Counts are printed inside each bar; whiskers are 95% intervals over cells. Workload counts differ by device, so the bars describe the configurations each device measured rather than an isolated architectural effect. The criterion is met on 128 of 447 forward-prefill and 409 of 1203 decode configurations, and no device clears half in both regimes. Configurations that fail the criterion are not shown to be equal; they are configurations where no fastest backend was established. Panel B pools configurations across GPUs separately for forward prefill and decode.

of each other has not been shown to differ, so every rate below is computed twice, over all shared cells and over cells separated on both devices.

The regime split. Table 2 pools the ten device pairs among A10, A100, L40, L40S and H100. Restriction pulls the two regimes apart. Forward prefill flips on 27 of 64 separated comparisons, interval [29, 53], while decode flips on 29 of 289, interval [5, 15]. Restricting each device pair to the backends both devices ran, and re-deriving winners and separation over that restricted set rather than filtering afterwards, moves prefill to 33 of 72 and changes decode to 27 of 289, 9.3%. Figure 2 also reports source-to-target transfer latency ratios: the median transferred choice is 1.002× the target's own best in prefill and 1.000× in decode. That distribution includes comparisons whose winner did not change, so it describes the cost of carrying a choice rather than the cost of a flip. The common-set restriction removes backends one device of the pair never ran on that workload, FlashAttention 3 among them, and it does not lower the prefill rate.

RTX 5090 comparisons under the tested build. The lower half of Table 2 repeats the calculation for the five pairs including the RTX 5090. The decode rate stays high, at 63 of 85, while forward prefill is 16 of 30. The gap between the two regimes is large, and the decode side coincides with an availability difference: in most of those comparisons

		all shared cells		separated both	
device set	regime	n	flip	n	flip [95% CI]
Amp/Ada/Hop	prefill	720	66.0%	64	42.2% [29, 53]
	decode	1916	52.1%	289	10.0% [5, 15]
5090 vs rest	prefill	360	65.8%	30	53.3% [30, 74]
	decode	959	78.4%	85	74.1% [56, 90]

Table 2: Winner-flip rate, pooled over device pairs. *Amp/Ada/Hop* pools the ten pairs among A10, A100, L40, L40S and H100; *5090 vs rest* the five joining the RTX 5090 to each. *n* counts device-pair comparisons on cells both devices ran, *flip* the share where the fastest implementation differs. The right columns keep only cells separated on both devices; their interval is a 95% cluster bootstrap over cells, 2000 draws, since one cell contributes to several pairs.

the other device's winner is FlashInfer, which produced no usable timing on the 5090 (Section 6). That is an observed association on this build. We did not measure what FlashInfer would have cost there, so we do not attribute the change to architecture or to availability as a cause.

6 Backend availability under the tested builds

FlashInfer produced no usable timing on RTX 5090. FlashInfer is the decode winner on 128 of 191 cells on A10, 133 of 192 on A100, 131 of 244 on H100 and 87 of 192 on L40, though only 49 of 192 on L40S, where the SDPA path leads instead. On the RTX 5090 it wins nothing because

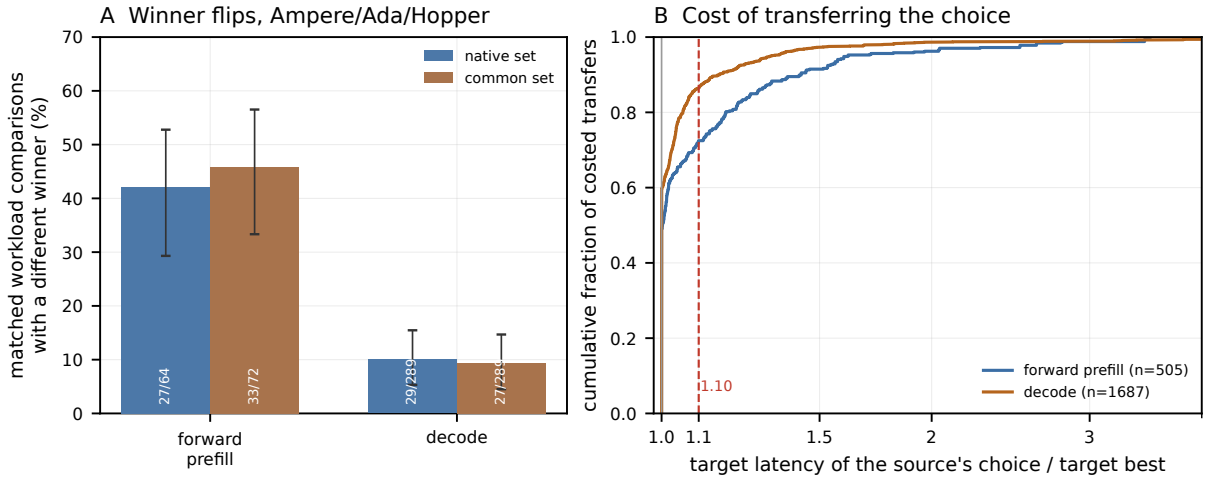

Figure 2: Winner-change rates and the latency cost of transferring a backend choice. *Left:* the five Ampere, Ada and Hopper GPUs, over each device's all backends each GPU ran and over the backends both devices of a pair ran on that workload, with winners and separation re-derived inside the restricted set. *Right:* all six GPUs, over ordered source-to-target pairs that have a separated winner on the source; the quantity is the target-device latency of the source's chosen backend divided by the target's own best. No eligible target timing existed for 85 of 590 forward-prefill and 287 of 1,974 decode source choices; those are counted, not costed. Winners differ on 27 of 64 forward-prefill and 29 of 289 decode comparisons, yet the median transfer costs 1.002× and 1.000×, with 95th percentiles of 1.61× and 1.32×. The distribution mixes comparisons whose winner changed with those whose winner did not, so it describes the cost of carrying a choice rather than the cost of a flip.

it never ran: of 960 attempted rows, 950 raised `RuntimeError: FlashInfer requires GPUs with sm75 or higher` across 190 distinct cells, and the other 10 hit the allocator. The text names sm75 on an sm120 part, so it is not a diagnosis, and how much it would have won by there cannot be measured. What can be measured is the fallback: the RTX 5090's decode winner is `flash_attn_with_kvcache` on 137 of 192 cells at a median 115.7 μs.

The wheels themselves. Table 4 is a coverage map, not a performance result. Its first two rows are the packaging asymmetry the prefill flip rates sit beside: Over forward-only prefill, FlashAttention 2 is the winner on 35 of 72 A100 cells, 13 of 72 on the RTX 5090, 2 of 72 on L40 and none on A10 or L40S; on H100 it wins none of the 87 while FlashAttention 3 takes 48. Counts pooling forward with forward-plus-backward differ substantially and are not used here. That pattern is *consistent with* the packaging but not shown to be caused by it: we did not rebuild flash-attn for the missing targets. FlashInfer is the other kind of gap: it JITs, so its coverage belongs to the toolkit on the host, not to the wheel. We report its sm_120 failure and not a cause: every other library in the audit carries native sm_120 SASS built under the same CUDA 12.8, so a blanket claim that the toolkit cannot target sm_120 is contradicted by the audit's own rows.

7 Threats to validity

Decode spans two harness revisions, and one host per device. The A10 and A100 decode rows carry harness commit `468b9a55` and every other decode row `91959d34`, recorded days later. Diffing the two revisions shows the measurement path is unchanged: `bench.py`, `impls.py`, `check.py` and the vendored Triton kernel are byte-identical, the image pins the same library versions, and the `run.py` differences decide which cells are attempted rather than how a cell is timed. The revision label therefore does not mark a change in what was measured, and we do not treat it as one. Driver and host are the real covariates, and they cannot be separated from device in this design: each device was measured on the host that had it. The L40 rows share driver 595.71.05 with A10 and A100 yet carry the later commit, and A10 against L40 flips on 0 of 36 separated comparisons while H100 against L40, one commit and two drivers apart, flips on 9 of 33. We report these as observations; the design does not separate architecture from driver and host, so we do not claim which of them the difference follows. A single-host replication of the whole grid would still be worth having, and we did not have six identical hosts. All six de-

vices did run prefill at one harness commit. A100 prefill was however re-measured on a rented host, driver 580.126.16, after the cluster run reached only one usable process repeat, so that device's two regimes come from two hosts. No comparison crosses the boundary, since every ranking is formed within one device and one regime, and the two hosts measured copy bandwidth within 0.05% of each other.

Repeat counts and comparison coverage. Across the dataset 1 of 11,835 usable triples has a single process repeat; its confidence interval is undefined and it cannot establish separation. Requiring a separated winner on both sides is a heavy restriction: it leaves 64 of the 720 forward-prefill and 289 of the 1916 decode comparisons among Ampere, Ada and Hopper devices, excluding 656 and 1627 respectively. That is the right restriction for a ranking claim, and it leaves the headline comparisons resting on tens to low hundreds of paired configurations.

Unprobed dispatch cells. 33.5% of `OK` attempts captured no kernel trace and 26 of 651 dispatch classes have no probed row at all, so the 0.449% mismatch rate is a rate among probed rows. On the unprobed rows the answer is unknown, not zero.

What the derived quantities do not establish. Effective KV bandwidth is an analytic byte count divided by measured latency, not measured DRAM traffic. Its ratio to the copy benchmark does not establish bandwidth saturation. Eager and CUDA-graph measurements use different timing routines, and a bare CUDA-event overhead is not a per-call resolution threshold. We therefore do not infer a timing floor from that comparison.

What was not measured. Every cloud host we rented blocked Nsight Compute and Nsight Systems, so no occupancy, cache-hit-rate or counter-derived roofline claim appears here, and every mechanism in Section 6 rests on a static binary audit and on run-time status codes. We measured exactly one torch, flash-attn and flashinfer combination, so a different wheel is a different experiment rather than a replication. What is timed is a single attention call on six devices from one vendor.

8 Conclusion

In the tested configurations, separated backend choices changed more frequently across Ampere, Ada and Hopper GPUs for forward prefill than for decode. Median transfer penalties were small, but the upper tail and missing target timings make the median insufficient for selecting a backend. These results support evaluating backend choices on the intended GPU and workload, with both uncertainty and execution coverage reported.

A Repeat-count sensitivity

L40 was given three planned process repeats against five elsewhere, and its median configuration keeps two, so its bootstrap intervals are the widest in the study. Dropping the device entirely moves the separation rate from 28.6% to 30.1% for forward prefill and from 34.0% to 34.2% for decode. Neither headline depends on it.

B Backend availability

Table 3 lists which comparisons were possible at all.

C Library builds and GPU support

The audit records which GPU targets the installed libraries were built for. It says what could have run, not why anything was fast: no rebuild for a missing target was performed, and CUDA permits a cubin built for a lower minor compute capability to run on a higher one within a major version.

References

Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, and 30 others. 2024. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.

Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM:

backend	regime	GQA	support	OK	unsup.	fail
D0-naive-kv	dec	both	YYYYYY	5530	0	10
D1-inductor	dec	expanded	YYYYYY	194	5346	0
D2-sdpa	dec	native	YYYYYY	5535	0	5
D3-fa-kvcache	dec	native	YYYYYY	5515	0	25
D4-flashinfer	dec	native	YYYYY.	4565	0	975
D6-fa-prefill-at-1	dec	native	YYYYYY	5515	0	25
P0-naive	pre	both	YYYYYY	3148	0	851
P1-inductor	pre	both	YYYYYY	3209	0	790
P1-inductor-nofuse	pre	both	YYYYYY	121	3873	5
P1-inductor-where	pre	both	YYYYYY	121	3873	5
P2b-sdpa-mem-eff	pre	native	YYYYYY	3969	30	0
P2c-sdpa-flash	pre	native	YYYYYY	3999	0	0
P2d-sdpa-cudnn	pre	native	YYYYYY	3999	0	0
P3-triton	pre	both	YYYYYY	3999	0	0
P4-fa2	pre	native	YYYYYY	3999	0	0
P4h-fa3	pre	native	..Y...	795	3204	0

Table 3: Which comparisons are possible at all. *support* is one character per device in the order A10, A100, H100, L40, L40S, RTX 5090: `Y` produced at least one usable row there, `.` attempted and produced none, `-` never attempted. *unsup.* counts configurations the backend declined in advance and *fail* runtime errors plus allocations refused or hit. FlashInfer reads `YYYYY.`: it ran on five devices and produced no timing on the RTX 5090. FlashAttention 3 reads `..Y...`, H100 only, by construction.

library	sm_80	sm_86	sm_89	sm_90	sm_120	PTX
flash-attn 2.8.3	SASS	*80*	*80*	SASS	SASS	none
libtorch_cuda	SASS	SASS	SASS	SASS	SASS	none
cuDNN engines	SASS	SASS	*86*	SASS	SASS	sm_120
cuBLAS	SASS	SASS	*86*	SASS	SASS	sm_120
flashinfer 0.6.18	JIT	JIT	JIT	JIT	none	n/a
flash-attn 3	n/f	n/f	n/f	n/f	n/f	n/f

Table 4: Binary provenance of the installed wheels, from `cuobjdump`, over the compute capabilities of the six devices. SASS means native machine code is present; *PTX* is the embedded PTX a driver could compile where there is no cubin. An italic number means none was shipped and the target runs that older architecture's cubin under CUDA minor-version binary compatibility. JIT means the library ships no cubins and builds at run time, *none* that this build produced no kernel there under CUDA 12.8, and *n/f* that the audit found no such library, which for FlashAttention 3, a path that ran on H100, is a gap in the audit and not in the install.

An automated end-to-end optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 578–594.

Tri Dao. 2024. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35.

Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. 2023. Flash-decoding for long-context inference. PyTorch Blog.

Tom Deakin, Simon McIntosh-Smith, James Price, Andrei Poenaru, Patrick Atkinson, Codrin Popa, and Justin Salmon. 2019. Performance portability across diverse computer architectures. In *2019 IEEE/ACM International Workshop on Performance, Portability and Productivity in HPC (P3HPC)*.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*.

Jianling Li, Shangzhan Li, Zhenye Gao, Qi Shi, Yuxuan Li, Zefan Wang, Jiacheng Huang, Haojie Wang, Jianrong Wang, Xu Han, Zhiyuan Liu, and Maosong Sun. 2025. TritonBench: Benchmarking large language model capabilities for generating Triton operators. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 23053–23066.

Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. 2025. KernelBench: Can LLMs write efficient GPU kernels? In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*.

Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative LLM inference using phase splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*.

S. J. Pennycook, J. D. Sewall, and V. W. Lee. 2016. A metric for performance portability. *Preprint*, arXiv:1611.07409.

Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, pages 10–19.

Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76.

Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and customizable attention engine for LLM inference serving. In *Proceedings of Machine Learning and Systems (MLSys)*.

Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. 2020. Ansor: Generating high-performance tensor programs for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 863–879.

Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.